



JavaFX多线程技术基础

北京理工大学计算机学院
金旭亮

如果不使用多线程.....



示例：UnresponsiveUI.java

```
view.changeFillButton.setOnAction(actionEvent -> {  
    final Paint fillPaint = model.getFillPaint();  
    if (fillPaint.equals(Color.LIGHTGRAY)) {  
        model.setFillPaint(Color.GRAY);  
    } else {  
        model.setFillPaint(Color.LIGHTGRAY);  
    }  
    try {  
        //这句将导致主线程无法及时响应用户的鼠标和键盘等操作  
        Thread.sleep( millis: 20000);  
    } catch (InterruptedException e) {  
    }  
});
```

在JavaFX应用程序线程中干太多事，
将会使应用程序失去响应。



利用多线程使应用程序保持响应



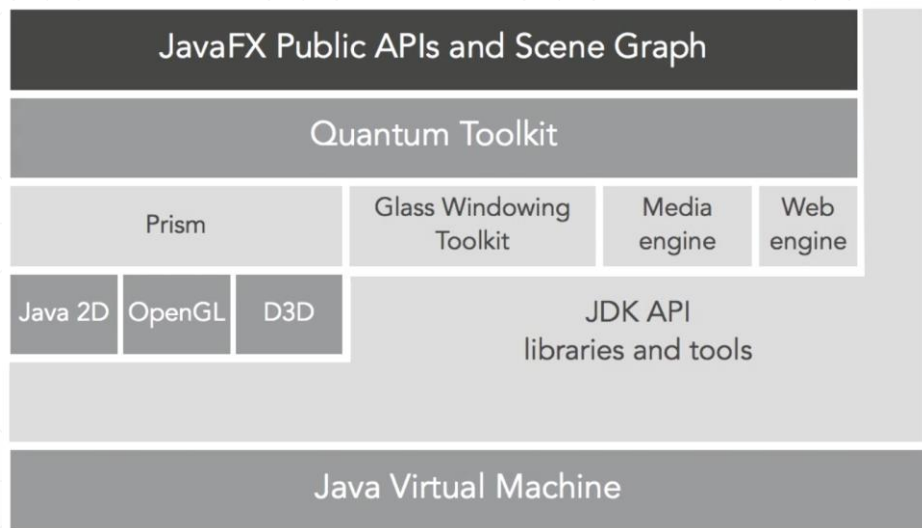
示例: ResponsiveUI.java



JavaFX不允许跨线程直接访问UI控件!

```
view.changeFillButton.setOnAction(actionEvent -> {  
    final Paint fillPaint = model.getFillPaint();  
    if (fillPaint.equals(Color.LIGHTGRAY)) {  
        model.setFillPaint(Color.GRAY);  
    } else {  
        model.setFillPaint(Color.LIGHTGRAY);  
    }  
    //在独立的工作线程中执行任务  
    Runnable task = () -> {  
        try {  
            Thread.sleep( millis: 3000);  
            //Lambda表达式执行的任务, 将会被推送到  
            //JavaFX application thread中执行  
            //因此, 里面可以有直接访问UI控件的代码  
            Platform.runLater(() -> {  
                final Rectangle rect = view.rectangle;  
                double newArcSize =  
                    rect.getArcHeight() < 20 ? 30 : 0;  
                rect.setArcWidth(newArcSize);  
                rect.setArcHeight(newArcSize);  
            });  
        } catch (InterruptedException e) {  
        }  
    };  
    new Thread(task).start();  
});
```

大致了解一下JavaFX技术架构-1



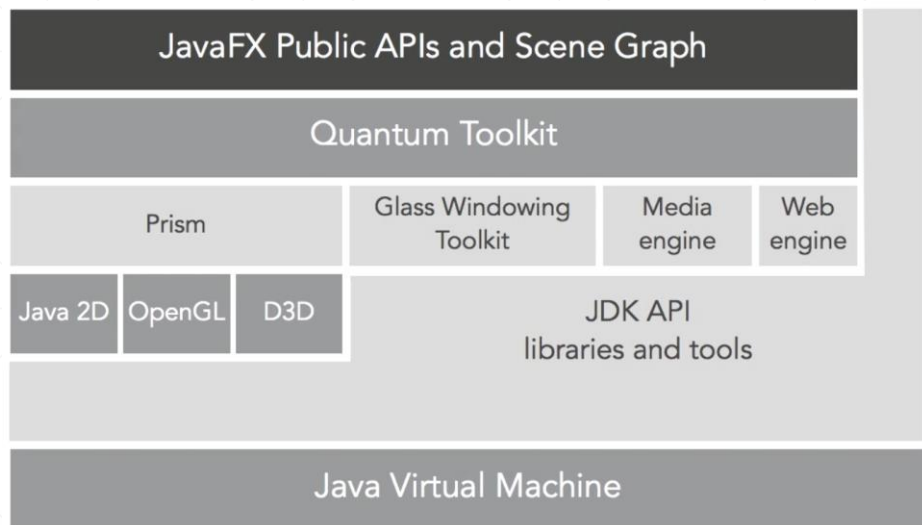
Prism是一个图形引擎，它会检测当前平台是否具备硬件加速能力，从而使用**DirectX**（windows）或**OpenGL**（Linux或Mac）获取较高的图形渲染性能，如果平台不支持，它就使用**Java 2D**，虽然性能差性，但兼容性好，总可用。

Prism使用专门线程完成图形渲染任务。

Glass Windowing Toolkit封装了本地操作系统中与Window和图形相关的系统服务（比如计时器），供上层组件使用。



大致了解一下JavaFX技术架构-2



Media Engine利用平台内置的编码/解码器，完成对于各种常见音频/视频数据的编码/解码任务，从而让JavaFX应用可以播放音视频文件。

Web Engine基于开源的WebKit，使用Prism完成网页内容的渲染任务。

Quantum Toolkit是一个抽象层，让上层JavaFX框架组件可以不必理会底层诸如Prism, Media Engine等的技术细节，就能提供JavaFX应用所需要的各种基础功能和支撑服务。

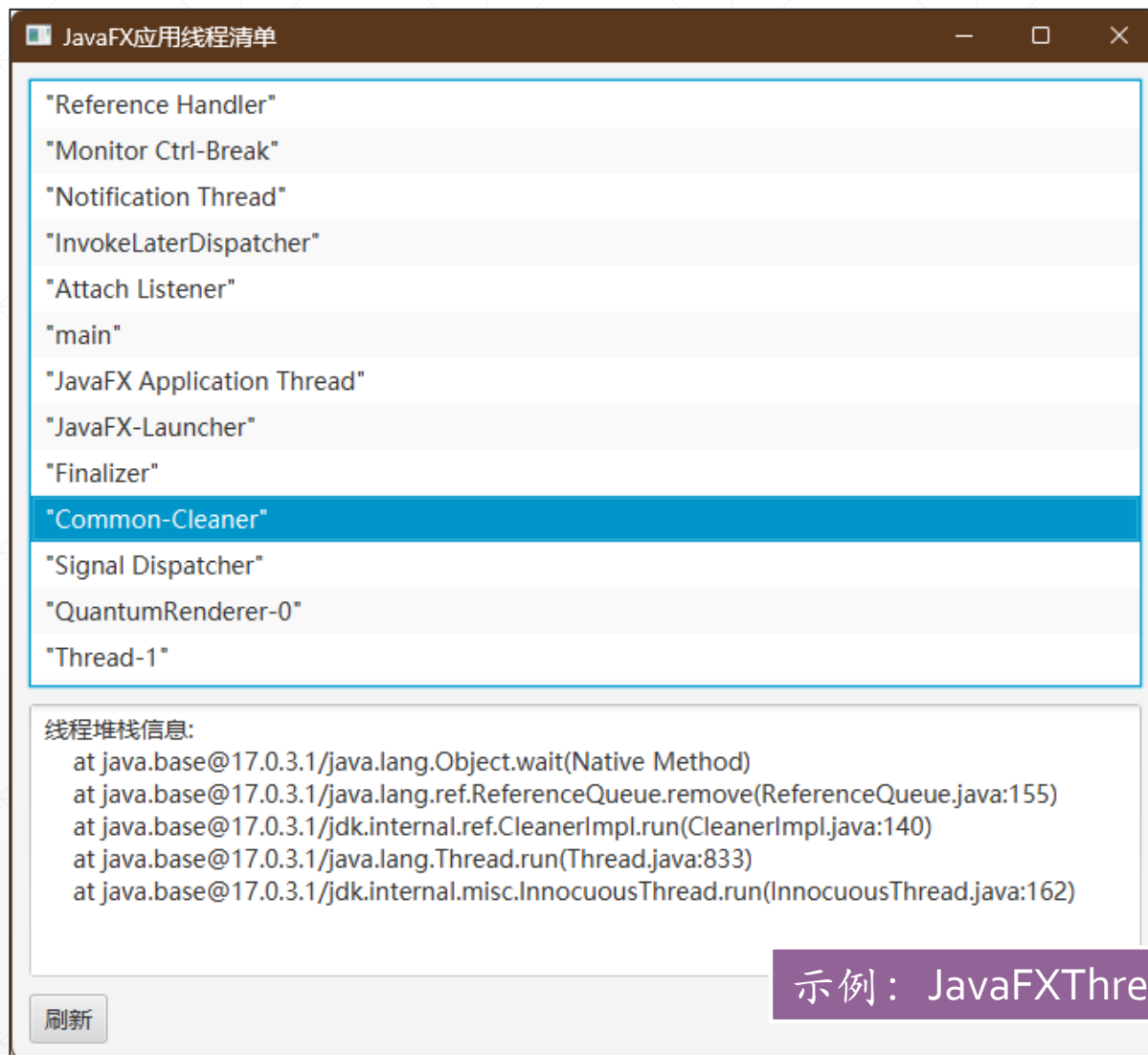
JavaFX中的主要线程



- ✓ **JavaFX应用程序线程**: 它维护UI控件的状态, 并且管理消息队列, 负责响应用户的操作, 修改Scene Graph, 重绘界面。此线程是JavaFX应用开发者需要关注的, 有时, 我们也把它称为“UI线程”或“主线程”
- ✓ **Prism渲染线程**: 此线程处理渲染工作, 使其与事件调度器独立开来。
- ✓ **多媒体线程**: 此线程会在后台运行, 通过使用JavaFX应用程序线程来在场景图中同步最近的帧。

底层线程,
普通开发者
可以不必理
会。

直观了解JavaFX应用中的各种线程

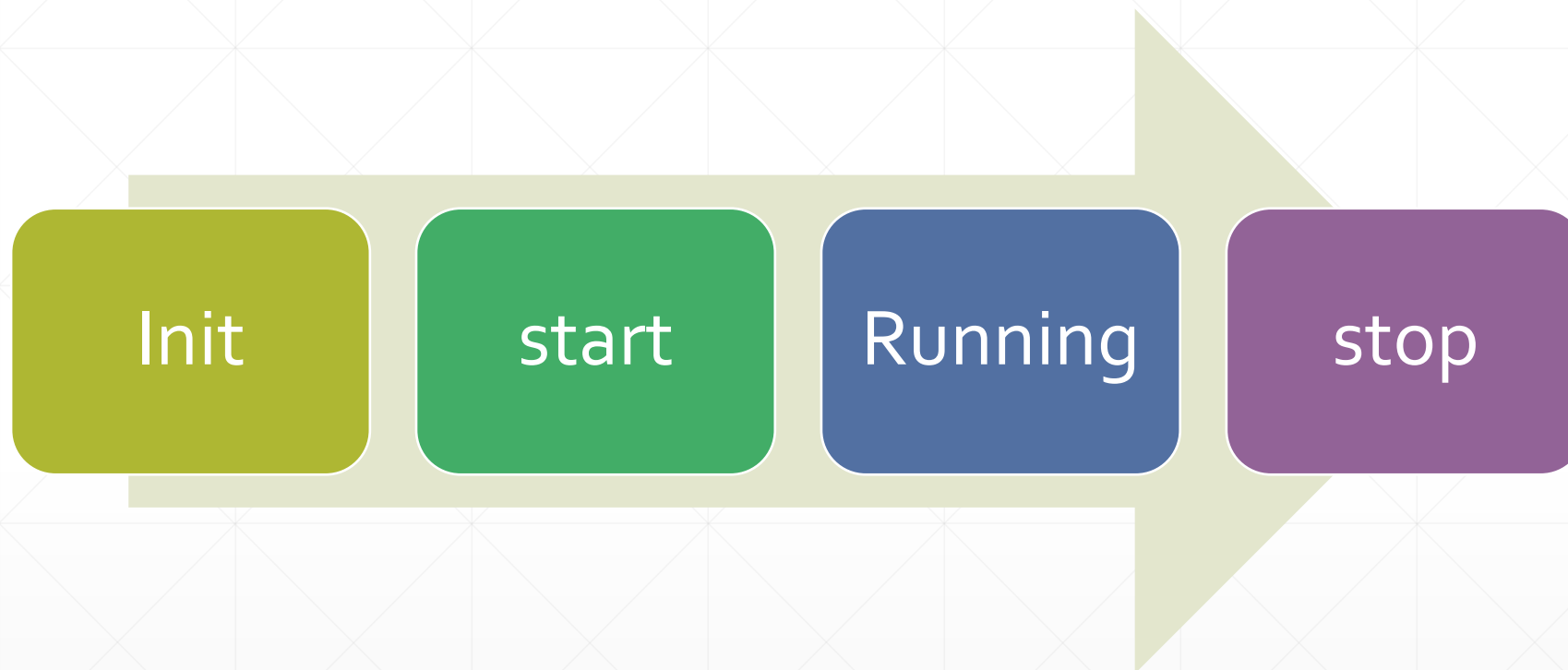
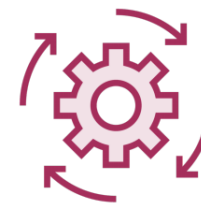


JavaFX应用在运行过程中，会启动多个线程，并且随着时间的推移，线程列表中的条目还会有变化。

详细介绍这些线程超出了本课程的范畴，对此，只需要有点感性的认识即可，不了解这些偏底层的细节，并不会影响日常开发。

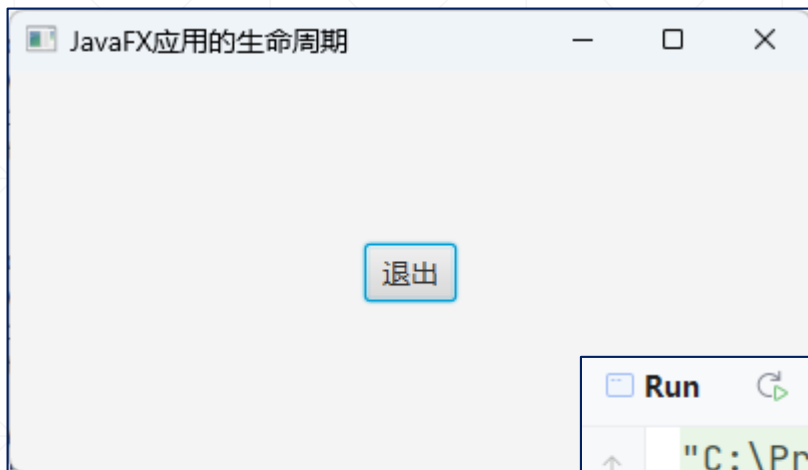
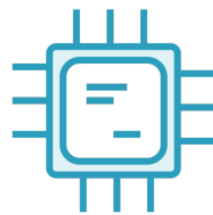
示例：JavaFXThreadsInfo.java

JavaFX应用的生命周期



在不同的生命周期阶段，JavaFX会回调不同的方法（这些方法定义于 `Application` 类中），而这些方法，又可以由不同的线程执行.....

JavaFX生命周期方法的调用线程



JavaFXLifecycleDemo.java

```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...
JavaFX Application Thread创建JavaFX Application实例:
JavaFX-Launcher 调用init()方法。
JavaFX Application Thread 调用start()方法
JavaFX Application Thread 调用stop()方法。

Process finished with exit code 0
```

了解JavaFX的线程分派策略



RunLaterApp示例运行截图

```
Run  [Run] [Stop] [Debug] [Profile] [More]
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
init():JavaFX-Launcher
start():JavaFX Application Thread
Runnable运行于Thread-3
```

示例代码



```
public class MainController implements Initializable {
```

2 usages

@FXML

```
private Button btnNewThread;
```

3 usages

@FXML

```
private Label lblInfo;
```

@Override

```
public void initialize(URL url, ResourceBundle resourceBundle) {  
    var threadName = Thread.currentThread().getName();  
    lblInfo.setText("控制器的initialize(): " + threadName);  
    btnNewThread.setOnAction(e -> {  
        newThread();  
    });  
}
```

1 usage

```
private void newThread() {  
    Runnable runnable = () -> {  
        var threadName = Thread.currentThread().getName();  
        System.out.println("Runnable运行于"+threadName);  
        Platform.runLater(()->{  
            lblInfo.setText("Label被更新于: "+Thread.currentThread().getName());  
        });  
    };  
    new Thread(runnable).start();  
}
```

从示例的输出可以看到，控制器的 `initialize()` 方法，由UI线程执行，而 `runLater()` 方法中的代码，则会被推送到UI线程中去执行。

关于Node对象的线程访问规则



当创建了一个Node对象，还没有将它追加到Scene Graph中去时，各个线程都可以随意地访问它。



但当此Node对象追加到Scene Graph之后，仅允许UI线程（即JavaFX Application Thread）访问它。